GENERATIVE AI & AGENTS

AI · ML · DS / REVISION NOTES

COURSE NOTES

Build a Language-Learning AI Agent

OpenAI, LangGraph, Ollama and the Model Context Protocol

9 SECTIONS

9 FIGURES

16 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

SECTION 5 Agent and LLM 3
6 ReAct agent basics 3
SECTION 10 Bob and GPT Model 9
SECTION 11 Advantage Signal and Reward Model 10
SECTION 12 Chain of Thought and Thinking Mode Fusion 11
SECTION 13 Reinforcement Learning 12
SECTION 14 ReAct Agent and Main 12
SECTION 17 Models and Ollama 14
SECTION 18 Main Ollama 15
SECTION 19 Clanki 15

1. First create a virtual environment with python 3.13 using `uv python install 3.13` and `uv init`.
2. Use the `https://github.com/eymenefealtun/all-words-in-all-languages` repo for data. You can use the kaggle datasets or UC Irvine datasets(https://archive.ics.uci.edu/datasets) also.
3. Create a folder `intelligent-learning-agent` and in that run `mkdir raw-word-lists` to create the folder.

Run the following code to copy the data.

```
cd "40.AI-Agent-Course-Build-a-Language-Learning-Agent-with-OpenAI-LangGraph-Ollama-MCP/language-learning-agent"
mkdir -p raw-word-lists
cp -r ../all-words-in-all-languages/{Catalan,Croatian,Danish,Dutch,English,Finnish,French,German,Greek,Italian,Polish,Portuguese,Romanian,Russian,Slovenian,Spanish,Swedish,Ukranian} raw-word-lists/
```

4. Install packages using `uv add pandas ipywidgets spacy spacy-transformers wordfreq python-dotenv typing-extensions langchain langchain_core langgraph langchain-openai langchain-ollama langchain-mcp-adapters matplotlib` in terminal under the `language-learning-agent` folder.

SECTION 5

Agent and LLM

First, run the code from clean-word-lists.ipynb to clean the data. Spacy is a family of libraries to process text data in natural languages like english. Here we use lemmatization. Spacy supports some languages which can be checked here `https://spacy.io/models` that's why we use some languages in above `cp` command.

Select the accurate model instead of efficient from here `https://spacy.io/models` because here accuracy matters.

We use zipf's law to find rare words and use wordfreq library to find rare words.

6 · ReAct agent basics

Here, we use English, Spanish and German languages for ReAct agent. Here we start to build language learning agent. Here, we build the langgraph react agent. Here we build the basic agent which take the prompt and return the list of random words.

7. What actually is an AI agent ?

An agent is essentially an LLM that has the ability to carry out multi-step processes and use tools to extend the capability of LLMs Basically LLM is only capable of input-output flow. We input a prompt and get a single output.

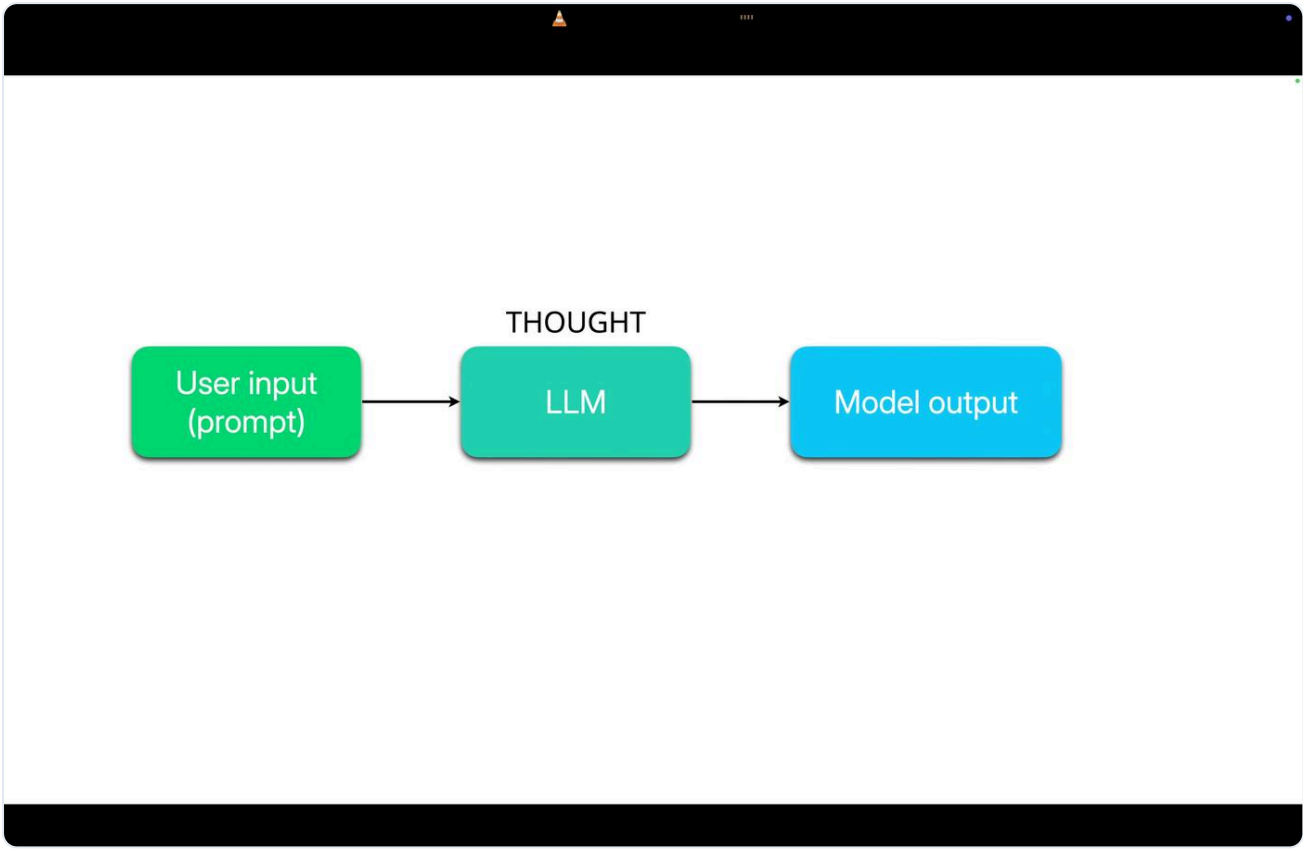


FIGURE 1.1

Agents extend this using what is called thought-action-observation cycle which allows LLMs to take more complex problems and solve them step by steps.

Agentic workflow start in a similar way to regular LLM use, in that users input a prompt. In the case of agents, this is usually a moderately complex problem or task. LLMs take the input and starts decomposing the problem into a series of smaller steps. The agent then works through each of these steps using the thought-action-observation workflow. For each step, LLM will think about what it could do to solve a problem, potentially using a variety of tools to achieve this.

Then it carries out an action. The agents store the results of this action in a memory and this memory state is passed back over to the LLM where it can observe the results.

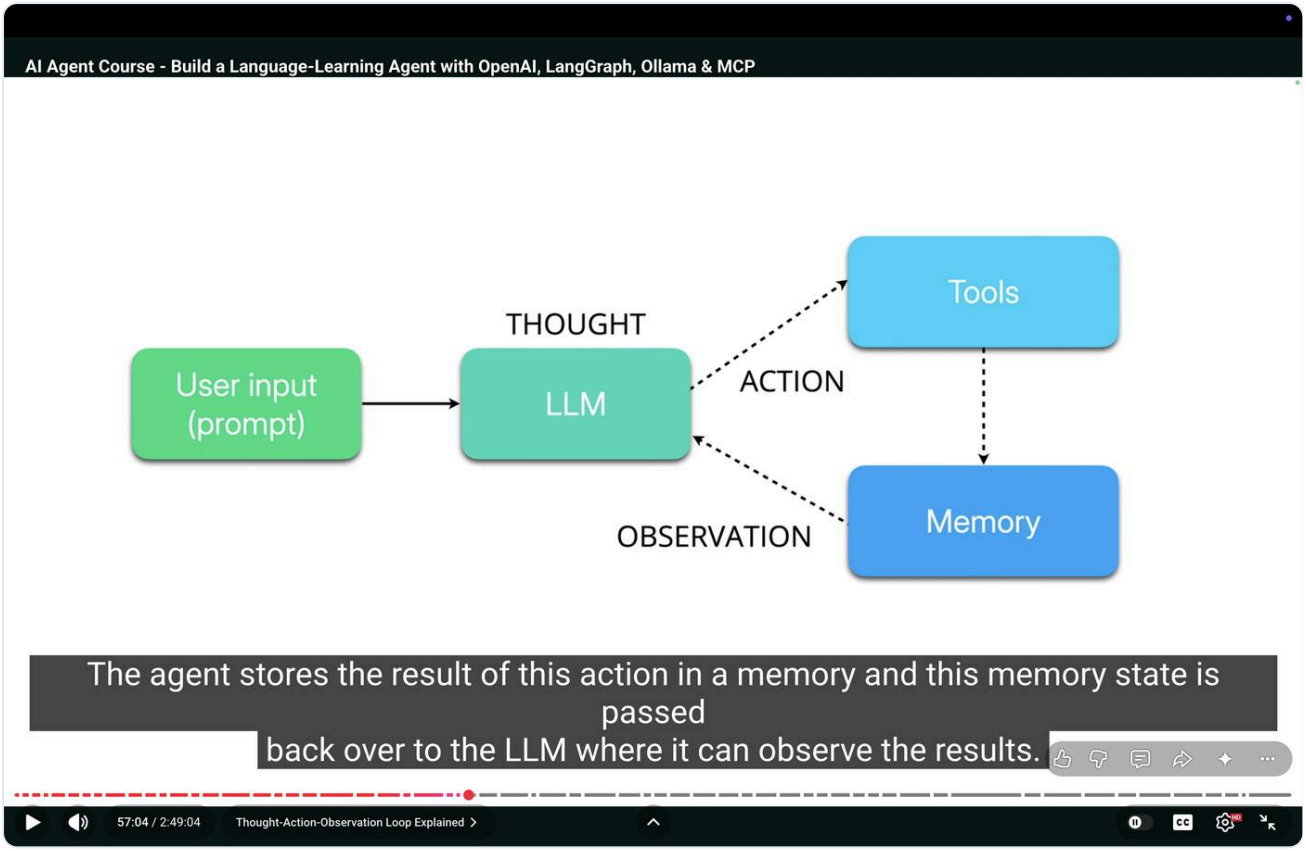


FIGURE 1.2

The LLM may then reattempt the same step if the results were not as expected means move on to the next step or if it judges that the goal has been achieved then it ends the agentic workflow.

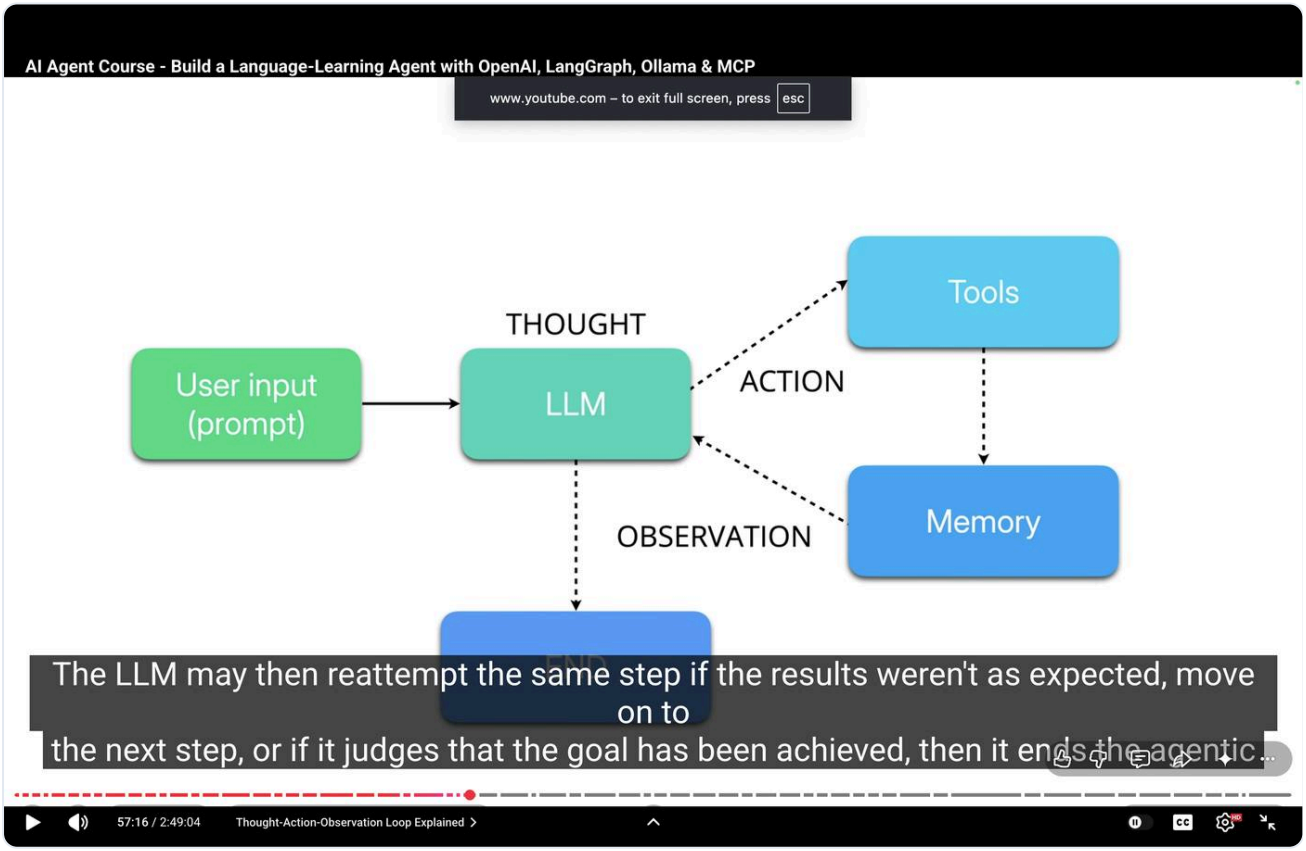


FIGURE 1.3

From this basic framework, you can build complex agents capable of doing many things that a base LLM can't.

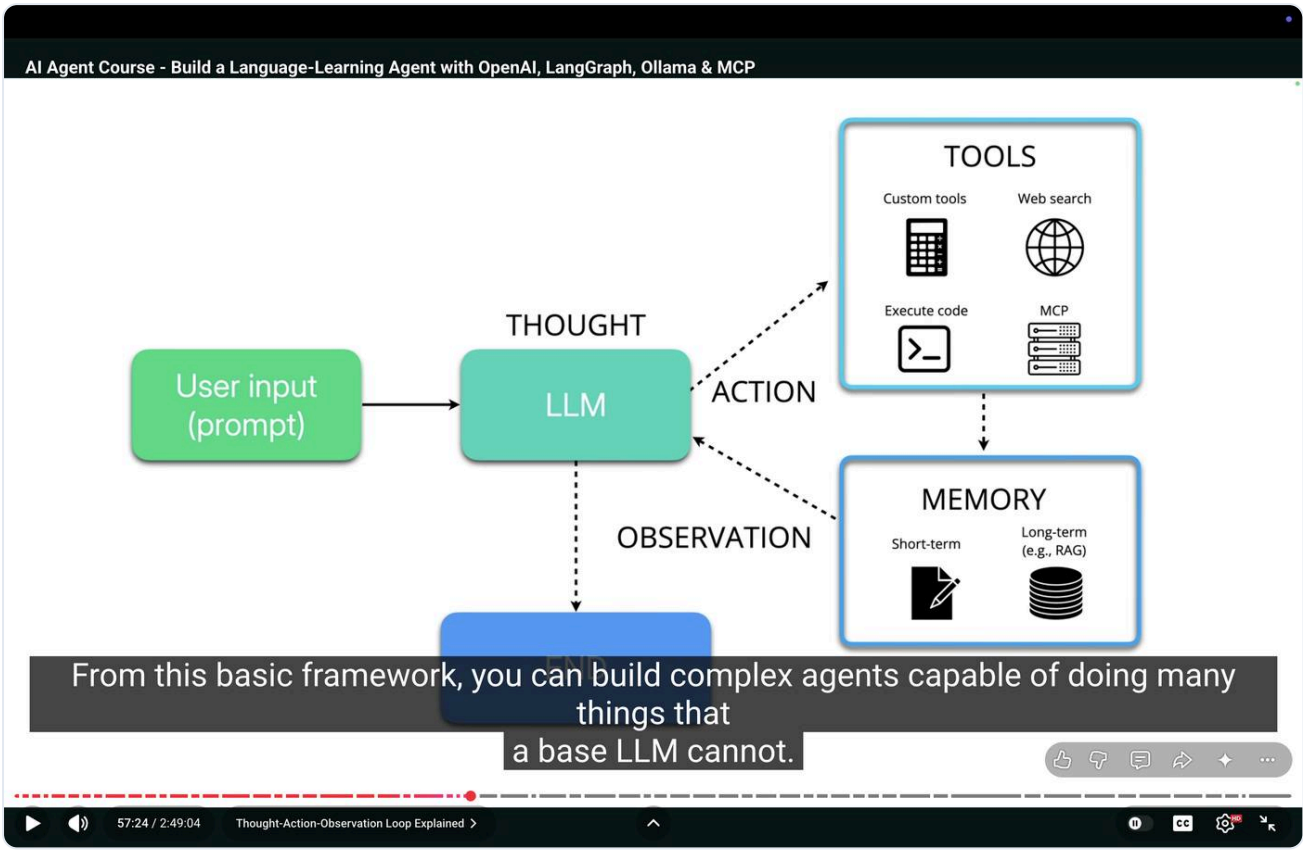


FIGURE 1.4

For example, you can give the LLM access to a wide variety of tools, which greatly expands what your agent can achieve. You can also create your own custom tools (for example, tools for performing calculations and we will build tools like this later on), you can give agent access to the web search, enabling them to retrieve up-to-date information. Also, they can interact with code execution environments in a controlled way. And agents can integrate with MCP servers via model context protocol (a standardize interface that lets them interact with many external tools and services in a consistent way).

Here, we are going to build a ReAct agent. React is short for Reasoning and acting and this type of LLM allows the LLM to decide which tool to use and in which order to carry out a goal. They tend to be suitable when an agent needs flexibility.

There are multiple different types of user requests coming in and agent can establish its own workflows using the available tools. Because getting an LLM to do problem decomposition and create an appropriate workflow is a complex task, React agents tend to work better for smaller and more focused use cases.

It is also possible to build much more structured agents.

Finally when you need an agent to do something really complex, you can consider building a multi-agent application. This allows you to build multiple smaller agents focused on more specific tasks and orchestrate them using a controlling agent.

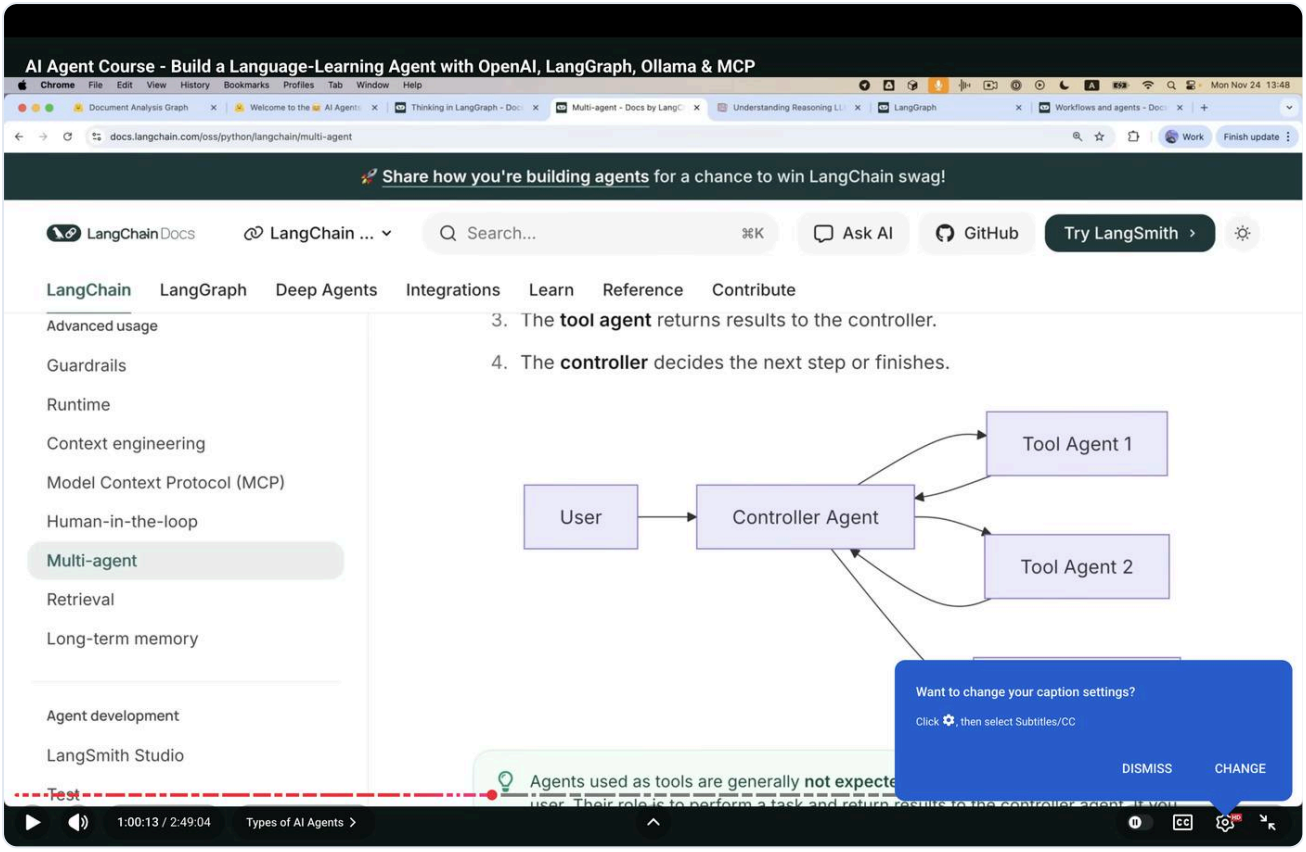


FIGURE 1.5

Multiagent applications can also help increase the security of your agentic applications.

So, Agent is a term used for a very broad family of applications that uses one or more LLMs to breakdown problems and use a huge variety of potential tools to carry out user tasks.

8. An LLM is basically a neural network that learned a lot about how language functions. For context based embeddings then advanced models come in which are based on transformers.

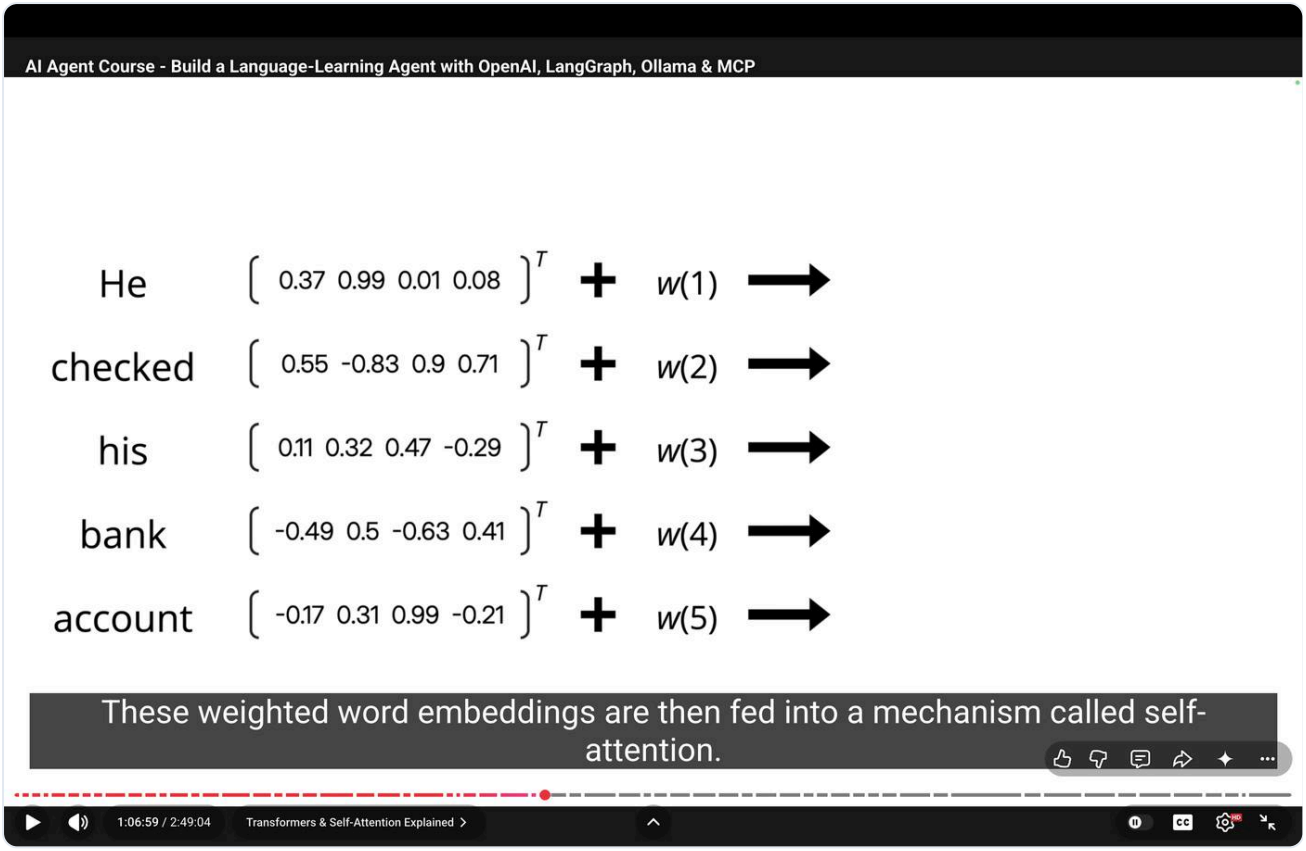


FIGURE 1.6

We make the weighted word embeddings by adding embeddings to positional weights like this is the first word(w(1)) and so on and then fed into self-attention mechanism. Self-attention basically uses the information from the weighted word embeddings to understand how much attention it needs to pay to other words in the sentence in order to get the meaning of that word.

The process of generating self-attention vectors and then normalizing them is called an encoder block.

9. GPT models are decoder based models. The most suitable LLMs for agents are reasoning models. So, how do we transform our base GPT models into one of these ?

There are multiple different ways of doing this but the explanation I will give here is how the Qwen3 reasoning models were trained. The base LLM goes through 4 post-training steps.

The first is called `Long chain-of-thought(CoT) Cold Start` training. The next stage is `Reasoning Reinfocement Learning(RL)`. Then the model undergoes `Thinking Mode Fusion` before finishing with a final `General Reinforcement Learning(RL)` stage.

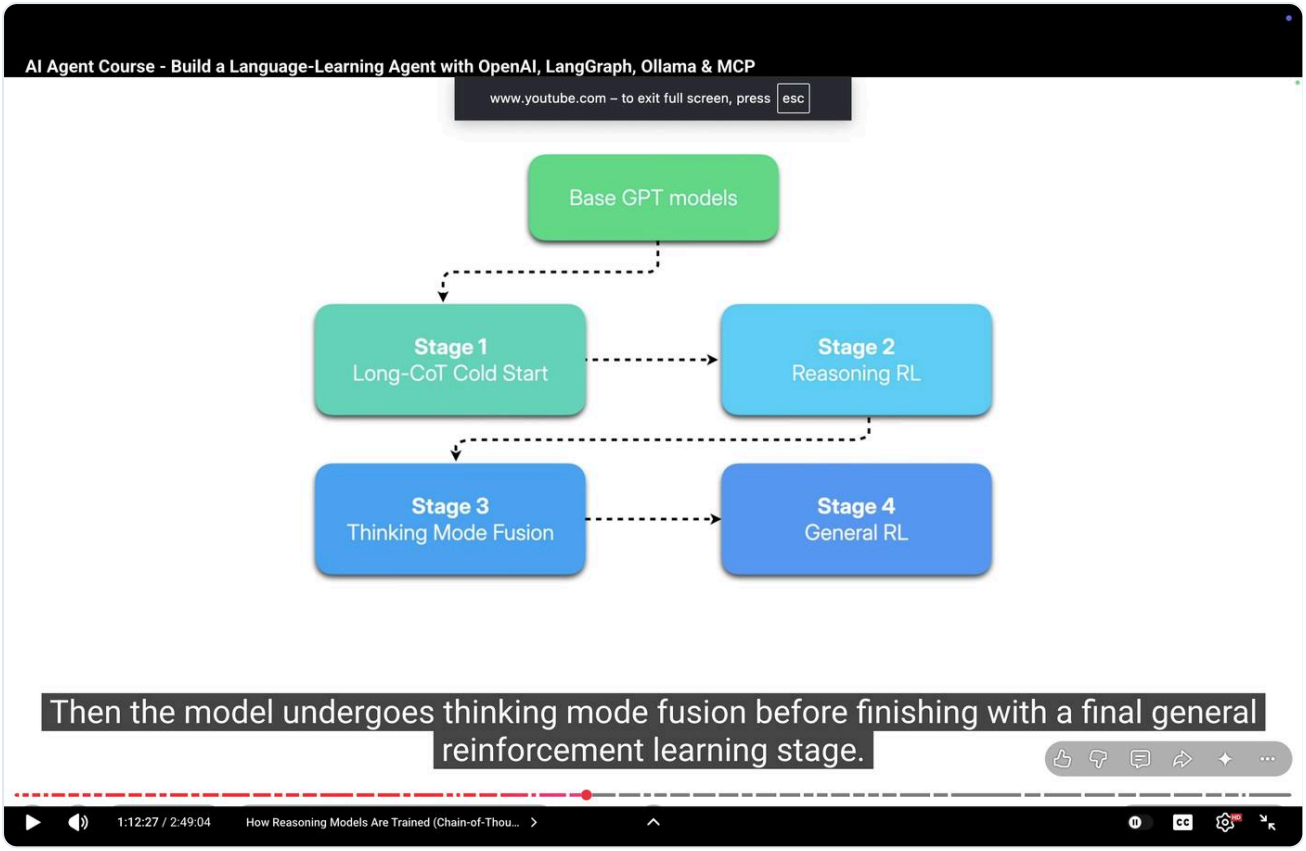


FIGURE 1.7

SECTION 10

Bob and GPT Model

Stage 1: Long chain-of-thought(CoT) Cold Start training:

When a GPT model is first trained, all it can do is generate natural text. It doesn't have any framework for problem decomposition that is taking a problem that requires multiple steps and breaking it down. So, this stage is about bootstrapping reasoning from scratch.

To do this, researchers create datasets where chain of thought reasoning is used to solve problems.

Example:

So, let's say we have a prompt:

Prompt:

Alice, Bob and Carol are sitting in a row.
Alice is not next to Carol.
Bob is not either end.
Who sits in the middle ?

Long-CoT:

There are 3 seats: Left, Middle, Right.
Bob is not either end --> Bob must be in the middle.
Now, check the constraint: "Alice is not next to Carol".
If Bob is in the middle, Alice and Carol must occupy the ends.
That satisfy the constraint automatically.
Therefore, Bob sits in the middle.

-- You can see how this is teaching the model to reason through each step of the problem to be more likely to reach a correct answer. Each example in the dataset is like this. All problems include a question and a detailed reasoning trace before the final answer.

The base GPT model is now further trained on this specific dataset, teaching it to output answers more like this.

This supervised fine-tuning stage is what gives the model its first thinking habit. It is still predicting the next token in these reasoning sequences but now it begins to internalize the structure of logical thought.

However, there is no guarantee that the conclusion it comes to are correct at this stage ot it will not get caught in a type of circular reasoning. So, the next stage is to teach it how to reason well. This is stage 2.

SECTION 11

Advantage Signal and Reward Model

Stage 2: Reasoning Reinforcement Learning(RL):

So, our fine-tuned model is: given a fresh set of problems and it generates multiple reasoning paths for each one. Some correct, some wrong like "who sits in the middle?" question, it can answer Alice. And these answers are used to train a separate reward model, which is able to predict things like the correctness, the logical consistency and the clarity of an answer.

So, once the reward model is trained, it can calculate an advantage signal for each new answer from the fine-tuned LLM. The advantage signal is how much better or worse than the expected answer, the predicted answer was. It then updates the weights of the LLM, pushing it closer to more correct and high quality answers and further away from incorrect, confusing or overly long answers.

Now, we have a model which is very strong in reasoning but not every problem requires deep, slow thinking.

So, in order to make the model more flexible, stage 3 i.e. Thinking mode fusion teaches it to distinguish problems that require deliberation (requires thinking) and those that can be solved directly (requires no reasoning).

SECTION 12

Chain of Thought and Thinking Mode Fusion

Stage 3: Thinking Mode Fusion:

Example:

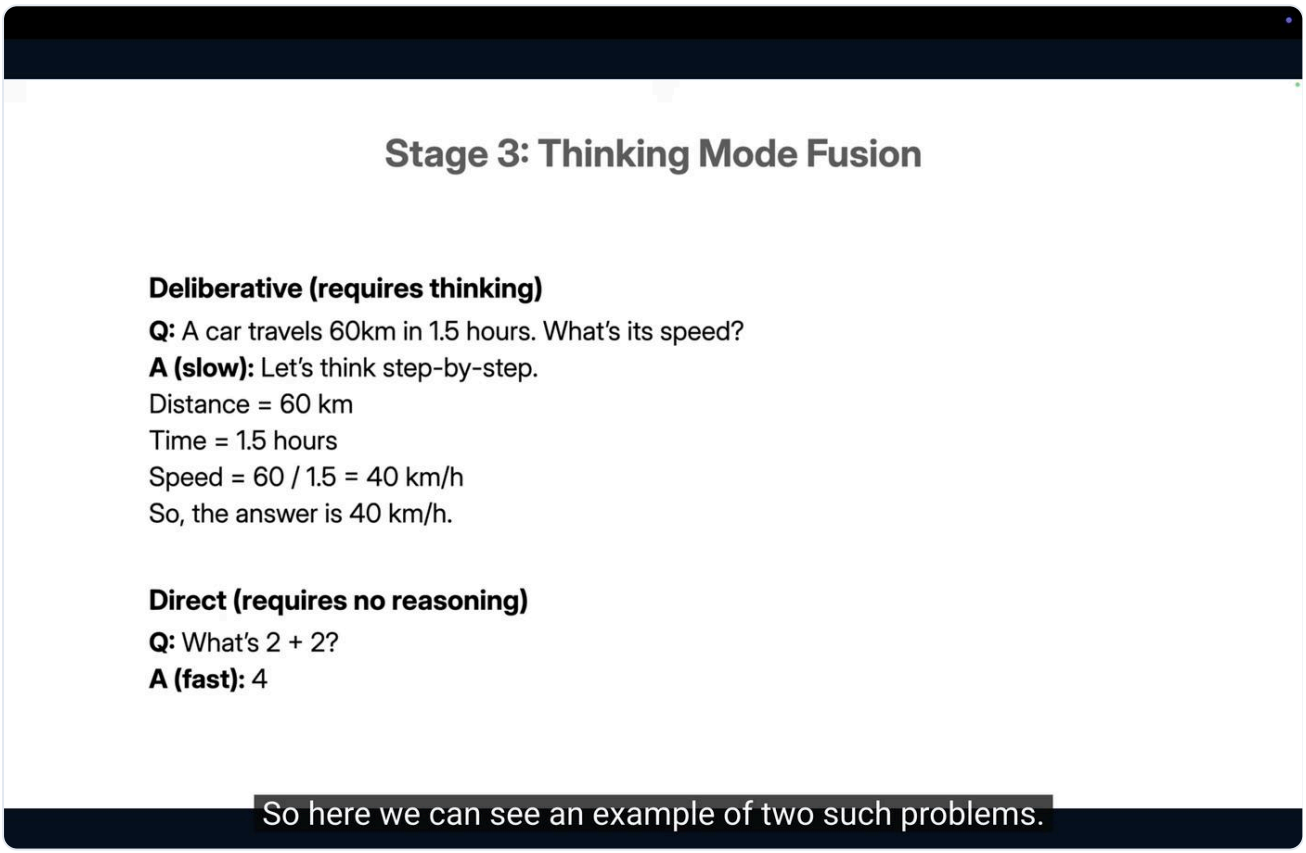


FIGURE 4.1

The first requires the chain of thought reasoning that the model has been taught so far. The second problem can be solved directly, meaning that applying deeper reasoning to solve it would be both a waste of time and compute. Keep in mind tha chain of thought reasoning uses many many more tokens than a direct problem solving.

There are a few different ways of doing thinking mode fusion, one is further fine-tuning the model. The dataset with a mixture of deep and direct problems is created and the model is trained to distinguish between when fast and slow reasoning is required.

SECTION 13

Reinforcement Learning

Stage 4: General Reinforcement Learning(RL):

And now the final step, general reinforcement learning. So, what we now have is a model that is highly specialized in reasoning but that's really all it can do. If you want to use it in a normal chat, it's likely to be unpolished, maybe even confusing and it's unable to do other important NLP tasks like summarizing documents and therefore final step uses reinforcement learning again to now teach the model to only apply reasoning when needed.

The model (qwen3) developers developed yet another dataset. This one containing a mixture of reasoning problems and other tasks such as instruction following, factual question and answering and creative writing.

As with the previous reinforcement learning step in stage 2, the LLM is given the same prompt from this dataset multiple times. This time, answers are scored based on their quality, a mixture of things like helpfulness, relevance, honesty, conciseness and reasoning quality. These ratings are done even by human raters or a more powerful LLM in a process called LLM-as-a-judge. And as with the last reinforcement learning step, these answer score pairs are used to train a second reward model, which can predict the overall quality of answers given by the model, not just its reasoning abilities. These scores are again used to update the model weights and answers that produce higher advantage signals.

SECTION 14

ReAct Agent and Main

So, after all that work, pre-training, fine-tuning and refining through reinforcement learning, we now have a model that is good at breaking down problems and working through them step by step and being flexible enough to do other things as well.

But are these models the best models ? Should we use them now for everything ?

Well, actually, No.

All of these post-training steps have forced the model into specializing in problem solving, meaning that it might not actually be the best model to use outside of settings where this is really important.

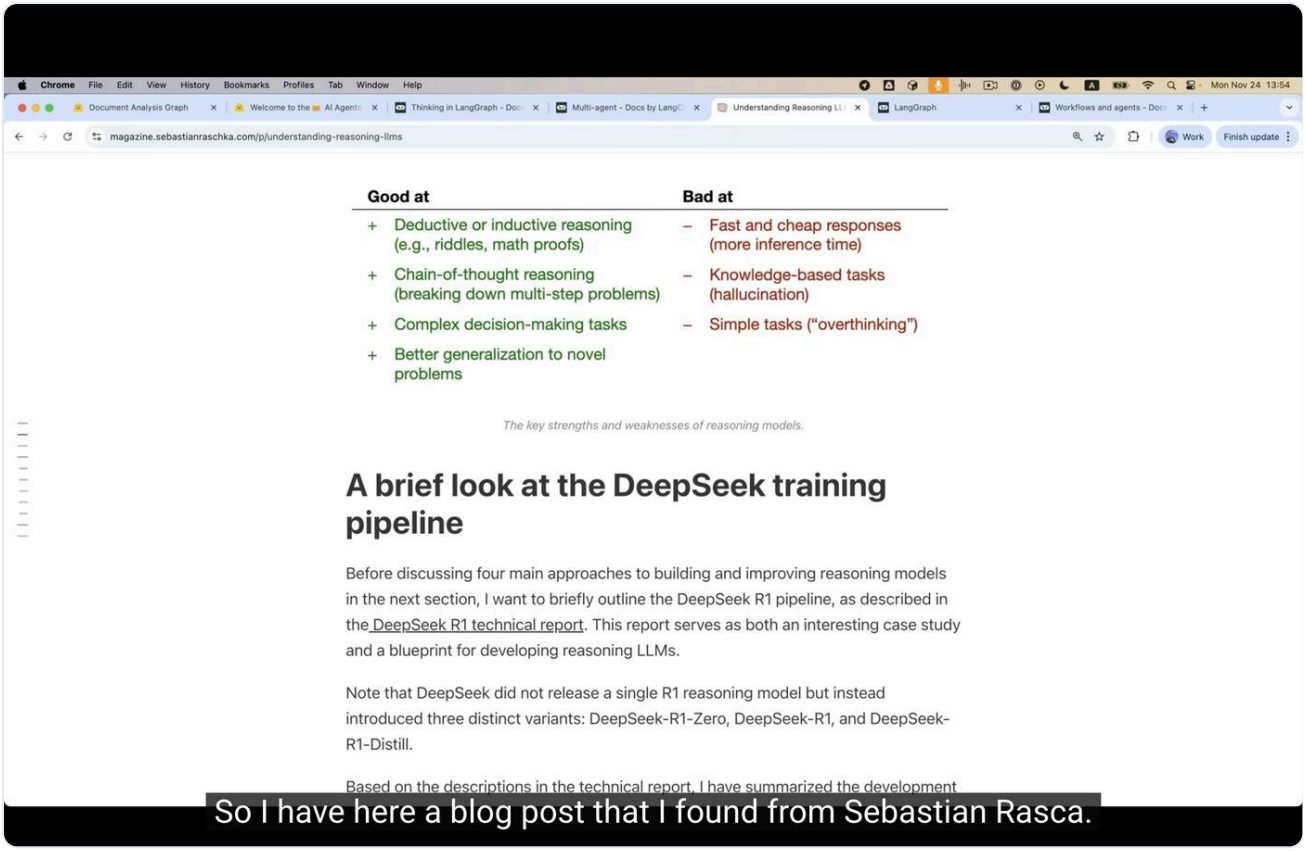


FIGURE 6.1

15. We can build many types of agents using LangGraph. You can build those more structured agents which require the LLM to only sort of make decisions at certain points. You can build multi-agent applications. Here, we build the ReAct agent.

Now, see the code of main.py.

In ReAct agent, we essentially allowing the LLM to do the planning and decision-making when it comes solving this problem.

16. Execute main.py file and you will get the following output:

┌─🍎 ankit@MacBook-Air 💻 .../AI-ML-DS/40.AI-Agent-Course-Build-a-Language-Learning-Agent-with-OpenAI-LangGraph-Ollama-MCP/language-learning-agent on main [✱3] 🐍 (.venv) 🐍 v3.13.14 is v0.1.0
└─ python3 main.py
Final messages: Here are 10 random words in German:

1. Verkehrsdrehscheibe
2. Kreisprozeß
3. Giftmischerei
4. Zentralkrankenhaus
5. Automobilmarke
6. überbiete
7. Managerin

8. Expertenrunde
9. Sonnenfest
10. Colonia

SECTION 17

Models and Ollama

Using Open-source local models:

So now we have basic agent which is running off the back of a proprietary model which is managed by a third party company.

And, while using these sort of proprietary models for agents is very convenient. They are very fast. They tend to be very powerful.

They do also come with a number of downsides. The first of these is cost. We need to pay for these models and because agentic application, in particular, tend to be very hungry. The cost can add up rather quickly. Another important consideration is data
privacy. Now, depending on the data that you are using in your agent, you may not want it sent to a third party provider, especially
if your contract for using the LLMs allows them to use your data for training the models.

And, a final consideration is using the right model for the job.

Using a smaller LLM that is trained for your specific use case has a number of advantages. It can save you in costs, in can save you in GPU utilization, and it can also make productionizing your agentic application much simpler.

Here, we use the Ollama. Ollama is a very popular tool for running LLMs locally. So, people might prefer to run LLMs locally for a number of reasons. It might be cost reasons, it might be the privacy reasons etc.

Ollama models are specifically designed to run locally, they tend to very lightweight models compared to these absolutely enormous closed source models or proprietary models that we were using in the previous section. These open source models have 3 billion or 7 billion parameters compared to 100 billion parameters of closed source models.

The other way in which ollama models are made smaller is through a process called quantization. So, this is basically where the model weights that is numbers that the model has learned during the training are reduced in precision. And, basically what it does is, it reduces the memory use and also speeds up the model, but it really tends to not impact the model performance that much.

- First download ollama from ollama.com/download for your operating system and then install it.

- Because we need a reasoning model then go to ollama site and navigate to the models page. Then filter on `Thinking` and see the models list.
- Here I settled on `qwen3` for this ReAct agent. Number of parameters of this model you choose depend on your machine.
- Using `ollama list`, check all the models which are installed in your vscode terminal.
- First, we need to install the model from the page `https://ollama.com/library/qwen3:8b` using command `ollama pull qwen3:8b` in the terminal and then run using `ollama run qwen3:8b` [5GB model].

SECTION 18

Main Ollama

Here, to use ollama, we are using main_ollama.py and agent_ollama folder for tools.py. Check the code and then run `python3 main_ollama.py` to see the output using qwen3 model. After that, we have used custom tools as well. Fo
r language translation tool, we are using llama3, so run it using `ollama run llama3` and also `ollama run qwen3:8b` in 2 different terminals and run the script `python3 main_ollama.py` in the third terminal.

LLMs are not deterministic, they do mistakes. So, double check everything.

SECTION 19

Clanki

Calling External tools through MCP:

MCP is a standardized protocol which makes communication between AI systems like AI agents and external tools much easier by standardizing those communications and breaking away from reliance on custom connections. These external tools are hosted on MCP servers. This is just a fancy way of describing any local program, any background service, any remote AI wrapper which communicates with AI agents via MCP.

There are an absolutely enormous number of MCP servers available. Some free, some paid. And these really help expand the range of
functionality of your AI agent. And then because we are talking of external tools, some of these carry security risks when used with
agents.

`https://mcpservers.org/` site has many many MCP servers and these falls under all sorts of different purposes, things like web scrapping, communication, productivity, development, etc. Check the site for more information.

For now, we are going to be using an MCP server called `Clanki` (`https://mcpservers.org/servers/jasperket/clanki`) which is going to allow us to create flashcards with a very popular piece of flashcard software called `Anki`. Both `Clanki` and `Anki` are free to use. Although Clanki says it is designed to use with Claude but it can be used with any LLM. So, we can use it with Qwen3 model as well.

Clanki has a quite large number of tools that allow us to interact with Anki. It allows us to create and manage Anki decks, create basic flashcards, create close cards, update existing cards, add and manage tags, view deck contents and information and it integrates with a plugin for Anki called AnkiConnect which we use here.

Installation:

- clone `git clone https://github.com/yourusername/clanki.git` where you want
- cd clanki
- npm install
- Run `npm run build` inside clanki directory

Now, inside `clanki` directory, go to `index.js` object and copy the `args` parameter value under `mcpServers`.

The next thing we need to create Anki flashcards is of course the Anki Application (`https://apps.ankiweb.net/`), this application is a free desktop application. You can download it from `https://apps.ankiweb.net/` and install it on your machine.

The final piece of setup we need to do is we need to add the AnkitConnect plugin to Anki. AnkiConnect is basically going to allow our agent to communicate with Anki. Run this app in the background when you have to run the code.

So, run the following commands in vs code terminal:

- git clone https://github.com/jasperket/clanki.git ~/Documents/clanki_setup/clanki
  cd ~/Documents/clanki_setup/clanki && npm install && npm run build
- go to `clanki ⟶ build ⟶ index.js` and use this path in `main_ollama.py`.
- You also need to setup AnkiConnect setup as well by the following process:

Installing the AnkiConnect plugin on your Mac takes just a few clicks. Open Anki, go to Tools > Add-ons > Get Add-ons..., enter the code 2055492159, and restart the application or don't restart if it is already working after running `python3 main_ollama.py`.

Now, see the output in Anki app.

20. For LLM tools, you can sandbox your agents so their access is limited and you can split tasks so that sensitive data and untrusted content are never mixing in the same step.